

NumPy: The Complete Master Guide

1. Introduction to NumPy

- **What is it?:** NumPy stands for *Numerical Python*. It is the fundamental library for scientific computing in Python.
- **Core Object:** The **ndarray** (n-dimensional array), which is a grid of values, all of the same type.
- **Why use NumPy instead of Python Lists?**
 - **Speed:** NumPy arrays are stored at one continuous place in memory, making them up to 50x faster than lists.
 - **Functionality:** Offers advanced mathematical functions for Linear Algebra, Statistics, and Random Number generation.

Setup:

Python

1. `import numpy as np`
-

2. Array Creation

A. Basic Methods

- **From Lists:** `arr = np.array([1, 2, 3])`
- **Zeros:** `np.zeros((3,3))` — Creates a 3x3 matrix of all zeros.
- **Ones:** `np.ones((2,4))` — Creates a 2x4 matrix of all ones.
- **Full:** `np.full((2,2), 7)` — Creates a 2x2 matrix filled with the number 7.
- **Identity Matrix:** `np.eye(4)` — A 4x4 matrix with 1s on the diagonal and 0s elsewhere.

B. Range-Based Creation

- **`arange()`:** Similar to Python's `range()`.
 - `np.arange(0, 10, 2) → [0, 2, 4, 6, 8]`
 - **`linspace()`:** Creates a specific number of samples over a specified interval.
 - `np.linspace(0, 1, 5) → [0. , 0.25, 0.5 , 0.75, 1. ]`
-

3. Array Inspection (Understanding your Data)

Before processing data, you must know its structure:

- `arr.shape`: Returns the dimensions (e.g., `(3, 4)` for 3 rows and 4 columns).
 - `arr.ndim`: Returns the number of dimensions (1, 2, or 3).
 - `arr.dtype`: Returns the data type of elements (e.g., `int64`, `float32`).
 - `arr.size`: Returns the total number of elements in the array.
-

4. Indexing and Slicing

- **1D Arrays**: Just like Python lists. `arr[1:4]` gets elements from index 1 to 3.
 - **2D Arrays (Matrices)**: Use a comma to separate row and column index.
 - `arr[0, 1]` → Element at 1st row, 2nd column.
 - `arr[:, 1]` → All rows, but only the 2nd column.
 - `arr[0:2, 1:3]` → Sub-matrix of the first 2 rows and columns 2-3.
 - **Boolean Indexing (Filtering)**:
 - `arr[arr > 5]` → Returns all elements greater than 5.
-

5. Mathematical Operations & Broadcasting

A. Element-wise Operations

You don't need loops. Operations are applied to every element automatically.

- `arr + 10` (Adds 10 to every element)
- `arr * 2` (Multiplies every element by 2)

B. Broadcasting

Broadcasting allows NumPy to work with arrays of different shapes during arithmetic operations. The smaller array is "broadcast" across the larger array so that they have compatible shapes.

6. Statistical and Mathematical Functions

NumPy makes statistical analysis very easy:

- `np.mean(arr)`: Average value.
- `np.median(arr)`: Middle value.
- `np.std(arr)`: Standard deviation.
- `np.sum(arr)`: Sum of all elements.
- `np.min(arr) / np.max(arr)`: Smallest and largest values.

The Axis Parameter:

- `axis=0`: Operation performed **Column-wise**.
 - `axis=1`: Operation performed **Row-wise**.
-

7. Array Manipulation

- `reshape(rows, cols)`: Changes the shape of an array without changing its data.
 - `transpose()` or `.T`: Flips the matrix (Rows become Columns).
 - `flatten()`: Converts a multi-dimensional array into a 1D array.
 - **Stacking**:
 - `np.vstack((a, b))`: Stack arrays vertically.
 - `np.hstack((a, b))`: Stack arrays horizontally.
-

8. Special Constants and Random Module

- `np.nan`: Represents "Not a Number" (used for missing data).
 - `np.inf`: Represents Infinity.
 - `np.random.rand(5)`: 5 random numbers between 0 and 1.
 - `np.random.randint(1, 100, 10)`: 10 random integers between 1 and 100.
-

Summary Table: NumPy vs Pandas

- **NumPy**: Best for numerical data, matrices, and complex math.
 - **Pandas**: Best for tabular data, CSVs, and data cleaning (uses NumPy internally).
- 2.